BIOMETRIC SECURITY · FACE LIVENESS · CASE STUDY

FaceGuard

Deepfake, face-swap and camera-injection defence layer for a **FaceTec**-based identity verification flow: vulnerability assessment, adversarial test campaign, AI mitigation layers and an operating policy.

FaceTec integration · Injection detection · Deepfake detection · PyTorch → ONNX · OpenCV · FastAPI · Prometheus · Grafana · ISO/IEC 30107-3 · CEN/TS 18099

Nonce photometric challenge
The screen flashes a per-session colour code. Live faces reflect it on time; injected video can't.

Deepfake CNN + forensics
PyTorch → ONNX, trained without a single deepfake (self-blended images). Catches unseen face swaps.

Calibrated fusion & policy
Approve · step-up · manual review · reject, with reason codes, alerts and a review queue.

vendor-only: ~~87.8%~~
6.4%
mean attack pass rate (APCER), 10 attack species

vendor-only: ~~94.7%~~
0.0%
stealth virtual-camera injection approved

vendor-only: ~~97.2%~~
0.8%
real-time face swap approved

honest users
2.40%
rejected (BPCER) · 6.8% stepped-up or reviewed

Held-out test split · 3610 attack sessions · simulated attack instruments on real public face photos

Built on my ML Model Serving Platform

Deliverable 1 — Vulnerability Assessment

FaceTec-based face verification flow vs. deepfakes, injection and presentation attacks

Scope	Remote identity verification flow built on FaceTec 3D liveness (web + native mobile), from camera to the onboarding / login decision
System assessed	*Reference* FaceTec integration (typical architecture described in §2). In a client engagement, §2 is replaced by the client's actual flow after code, config and log review (checklist in §7).
Method	Threat modelling (STRIDE on the data flow), attack-tree enumeration, rating per §1.3, validation of the attack classes that can be simulated in the FaceGuard harness (Deliverable 2)
Standards vocabulary	ISO/IEC 30107-1/-3 (presentation attack detection, APCER/BPCER), CEN/TS 18099:2024 (biometric data injection attack detection: injection attack *instrument* vs *method*), ISO/IEC 25456 (injection attack detection, in development)

> **Honesty note.** FaceTec is a proprietary SDK and no FaceTec licence was used for this portfolio project. This assessment draws on FaceTec's public documentation, how FaceTec flows are usually integrated, and the generic deepfake/injection threat class. It does not report any FaceTec zero-day. To the author's knowledge, at the time of writing there is no public, independently verified bypass of *current* FaceTec 3D liveness. The highest risks below are **integration gaps and attack classes outside ISO/IEC 30107-3 PAD certification**, not a flaw in FaceTec's matcher.

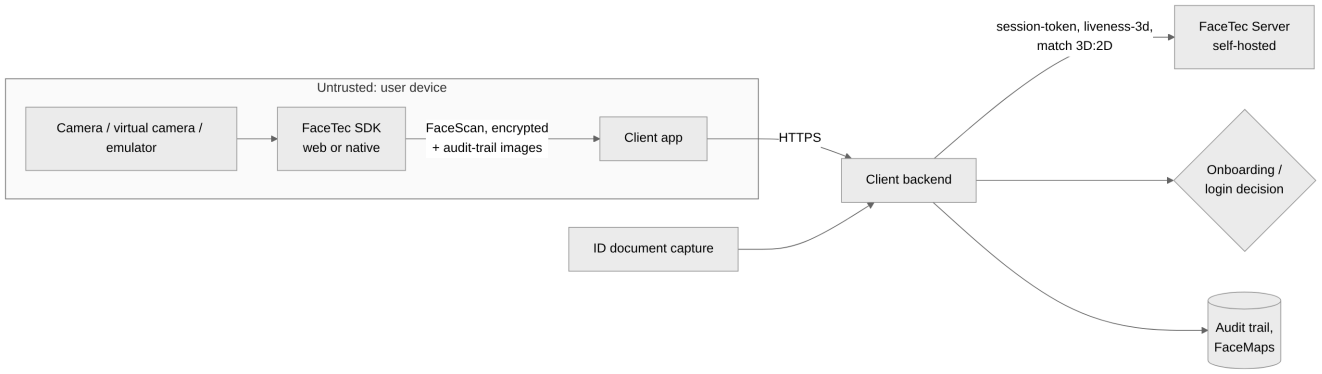
---

1. Threat model

1.1 Attacker goals

Goal	Typical attacker	What they need to beat
G1 Account takeover: pass re-authentication as the victim	Fraud ring holding the victim's photos or videos (social media, leaks, earlier KYC selfies)	Liveness + 1:1 match against the victim's enrolment
G2 New-account fraud: onboard with a stolen or synthetic identity	Fraud ring, money-mule recruiter	Liveness + 1:N/1:1 match against an ID document photo they control
G3 Account farming: many accounts, few real people	Bonus-abuse or bot operator	Liveness at scale; velocity and device-reuse controls
G4 Probing: learn where thresholds sit	Any of the above	Unlimited retries; verbose error messages

1.2 Data flow and trust boundaries (reference integration)



Trust boundary 1 is the device: everything it sends, including camera frames, device metadata and "success" callbacks, is attacker-controlled. *Trust boundary 2* is the backend ↔ FaceTec Server link. It is trusted, but only if the backend actually makes the call and reads the response correctly.

1.3 Rating method

- **Likelihood (1–5)** combines attack potential (ISO/IEC 18045-style: elapsed time, expertise, knowledge of the target, equipment) with tool availability. Commodity, tutorial-level tooling (OBS Virtual Camera, open-source real-time face-swap, Android camera-hook modules) raises likelihood.
- **Impact (1–5)** is the business effect of one success (ATO = 5, new-account fraud = 4, farming = 3).
- **Risk = L × I**: Critical ≥ 16 · High 10–15 · Medium 5–9 · Low ≤ 4.

---

2. Reference flow (what is being assessed)

1. The backend gets a **session token** from FaceTec Server and passes it to the client.
2. The FaceTec SDK runs the 3D liveness UX (the user moves closer to and further from the camera) and produces an **encrypted FaceScan** plus **audit-trail images**.
3. The client posts FaceScan + audit trail to the backend. The backend calls FaceTec Server (liveness / enrolment), then matches the user to the ID-document portrait (3D:2D) or to the enrolment FaceMap (3D:3D).
4. The backend returns FaceTec's result blob to the SDK to finish the UX, and stores the outcome, audit images and FaceMap.

FaceTec states its own coverage across five attack levels: Level 1–3 presentation attacks (photos and screens up to high-end 3D masks), **Level 4 tampering with the biometric payload**, and **Level 5 virtual-camera / video-injection "camera sensor bypasses"**. It also states that ISO/IEC 30107-3:2017 did not account for Level 5 attacks [R1].

---

3. Attack vectors

Legend for the *Mitigation* column: **FG-x** is the FaceGuard component that addresses the vector (Deliverable 3); **Ops** means an operational or process control (Deliverable 4).

3.1 Presentation attacks (artefact shown to a real camera)

ID	Vector	L	I	Risk	Native coverage	Residual gap	Mitigation
AV-01	Printed photo / paper cut-out of the victim	4 → 1*	4	Low (4)	Strong: 3D liveness, iBeta PAD testing	Very low	FG-PAD texture layer as an independent second opinion on the audit image
AV-02	Screen replay (phone, tablet, monitor)	4 → 2*	4	Medium (8)	Strong	Low. Lower-resolution camera filming a high-resolution display is the hardest case [R1]	FG-PAD (moiré / spectrum), FG-CI photometric challenge (a screen reflects uniformly)
AV-03	3D masks (latex, resin, silicone)	2 → 1*	5	Medium (5)	Strong for Level 2, claimed for Level 3	High-end masks cost money and time, so likelihood is low	Physical lab testing only (iBeta Level 2); out of scope for a software harness

* Raw likelihood → effective likelihood once FaceTec's native 3D liveness / PAD is in place. Risk is computed with the effective value.

3.2 Injection attacks (the camera is bypassed)

ID	Vector	L	I	Risk	Native coverage	Residual gap	Mitigation
AV-04	**Virtual camera** (OBS, ManyCam, e2eSoft…) feeding a **pre-recorded genuine video** of the victim into a **web** flow	4	5	**Critical (20)**	Camera-feed integrity checks (vendor-reported)	Browsers expose only a camera *label*, which can be renamed. Web is the weakest surface	FG-CI rules (driver signatures), **nonce-bound photometric challenge**, frame timing, sensor-noise checks
AV-05	**Real-time face swap** (DeepFaceLive / Deep-Live-Cam class) of a live attacker → virtual camera	3	5	**High (15)**	3D liveness requires a consistent 3D head across the zoom motion, which strains 2D swaps	Swap quality improves monthly. The attacker *is* live, so liveness signals stay genuine	FG-DF deepfake CNN, **challenge-reflection latency** (swap pipelines add 100–300 ms), FG-CI
AV-06	**Offline deepfake / face-swap video** injected	3	5	High (15)	As AV-04	Same as AV-04, plus no live subject	FG-DF, FG-CI challenge (the video cannot contain this session's nonce)
AV-07	**Fully synthetic face** (GAN or diffusion "synthetic identity") injected, paired with a forged ID document	3	4	High (12)	Liveness on an injected stream	Defeats 1:1 matching by construction, because the forged ID carries the synthetic face	FG-CI challenge, FG-DF, document-side forensics (Ops)
AV-08	**Mobile emulator / rooted device** with camera API hooks (Xposed/LSPosed "VCAM" modules, Frida, Magisk)	3	5	**High (15)**	SDK device checks	Root hiding (Zygisk, Shamiko) defeats naive checks	**Play Integrity / App Attest server-side verification**, FG-CI rules, challenge

ID	Vector	L	I	Risk	Native coverage	Residual gap	Mitigation
AV-09	**Challenge-aware injection:** a white-box attacker screen-captures the on-screen challenge and re-lights an injected stream in real time	2	5	Medium (10)	–	Residual. Detectable only through pipeline latency and physical plausibility	Short TTL, randomised slot timing, latency window, device attestation, native-only capture for high-risk actions

3.3 Integration and logic flaws (most frequently found in real integrations)

ID	Vector	L	I	Risk	Typical root cause	Mitigation
AV-10	**Client-side verdict trust:** the backend accepts `success: true` or `livenessPassed` from the app or SDK callback	3	5	**High (15)**, Critical where present	The backend proxies FaceScan, but the decision is read from the client's response	FG adapter: the verdict comes **only** from the server-to-server response, and client claims are ignored (tested)
AV-11	**Partial response checking:** only `success` / `wasProcessed` read; individual security checks (session token, replay, audit-trail verification) ignored	3	4	High (12)	Copy-paste from a sample	FG adapter requires *every* security check. Missing check = fail closed (tested)
AV-12	**API-level replay:** a valid captured submission (FaceScan + audit trail) is re-posted, or raced twice	2	5	Medium (10)	Session token not single-use or not bound to the user; no atomic consumption	FG sessions: HMAC token bound to session + nonce, **atomic one-time consumption**, 120 s TTL (tested)
AV-13	**Retry / probing abuse:** unlimited attempts let the attacker iterate until a sample passes	4	4	**Critical (16)**	No velocity limits; detailed error reasons returned to the client	FG velocity limits (identity / device / identities-per-device). Generic client errors; reasons only in back-office
AV-14	**Weak fallback path:** when FaceTec "fails", the user can upload a selfie or go to a lighter check	3	5	**High (15)**	UX pressure to reduce drop-off	Fallback = STEP_UP to a *stronger* channel (native app with attestation, NFC chip read, video call), never weaker
AV-15	**Reference poisoning:** the 3D:2D match is made against an ID-document portrait the attacker controls (forged, morphed, deepfaked)	3	5	**High (15)**	Liveness proves a live (or injected) face, not *whose*	Document forensics / NFC eMRTD chip portrait (signed), morph detection, 1:N duplicate search (Ops)
AV-16	**Account farming:** one device or one face enrols many identities	4	3	High (12)	No device-reuse or face-dedupe controls	FG `DEVICE_MULTI_IDENTITY` rule, 1:N face dedupe at enrolment (Ops)
AV-17	**Biometric data exposure:** audit-trail images and FaceMaps retained indefinitely or logged; leaked media feeds AV-04	2	4	Medium (8)	Over-retention, verbose logging	FG adapter drops `scanResultBlob` from logs. Retention policy and encryption at rest (Ops)
AV-18	**SDK / server drift:** old SDK builds in the wild without the newest injection checks; server version mismatches	3	3	Medium (9)	Slow app-update adoption	Minimum-SDK-version enforcement server-side; version label in metrics (Ops)

ID	Vector	L	I	Risk	Typical root cause	Mitigation
AV-19	**Coerced or recruited genuine user** (money mule performs liveness live)	3	4	High (12)	Not a liveness problem: the face *is* live and *is* the person	Behavioural and transaction analytics, document-to-person consistency, review of high-risk cohorts (Ops)

3.4 Risk heat map

```
              Impact →  3              4                        5
Likelihood 4            AV-16          AV-13                    AV-04
     ↓     3            AV-18          AV-07 AV-11 AV-19        AV-05 AV-06 AV-08 AV-10 AV-
14 AV-15
           2            .              AV-02 AV-17              AV-09 AV-12
           1            .              AV-01                    AV-03
```

---

4. Known risks of FaceTec and similar liveness SDKs against deepfakes

1. **Certification gap.** iBeta / ISO/IEC 30107-3 testing measures *presentation* attacks: something shown to a camera. It does not measure *injection*, where the camera is bypassed. A "Level 1 & 2 compliant" badge says nothing about AV-04…AV-09. CEN/TS 18099:2024 is the first standard specifically for injection-attack detection (instrument × method, evaluated by labs such as Ingenium [R2]), and ISO/IEC 25456 is in development.
2. **Web is the weakest surface.** Inside a browser the SDK cannot attest the camera hardware. It sees a `MediaStreamTrack` with a label and settings, both under attacker control. Injection risk on web is materially higher than in a native app with hardware attestation.
3. **Liveness ≠ identity.** A perfect liveness check on a *real-time face swap* confirms that a live human is present (the attacker) while showing the victim's face. This is why matching scores cannot be a liveness signal: in Deliverable 2 the face-swap species is measured with IAPMR.
4. **Signals that can be bought or rented.** "KYC bypass" services that sell virtual-camera + face-swap kits, plus leaked selfie/ID packs, keep lowering the attack potential of AV-04…AV-06.
5. **The integration is the vulnerability more often than the SDK.** In practice, AV-10…AV-14 (client trust, partial checks, replay, probing, weak fallbacks) are found far more often than SDK bypasses. They are cheap to fix and should be fixed first.

---

5. Prioritised findings and remediation roadmap

Priority	Action	Closes	Effort
P0, week 1	Decision only from the server-to-server FaceTec response. Require *all* security checks. Fail closed	AV-10, AV-11	Hours
P0	One-time, user-bound session tokens with atomic consumption. TTL ≤ 2 min	AV-12	Hours
P0	Velocity limits per identity, device and identities-per-device. Generic client-side errors	AV-13, AV-16	1 day
P0	Remove weaker fallbacks. Failure routes to STEP_UP in a *stronger* channel	AV-14	Product decision
P1, weeks 2–4	Capture-integrity layer: driver/emulator/hook/attestation rules + nonce photometric challenge + timing/noise model	AV-04, AV-06, AV-07, AV-08	FaceGuard FG-CI

Priority	Action	Closes	Effort
P1	Deepfake/face-swap detector on audit-trail images and capture frames, with a manual-review band	AV-05, AV-06, AV-07	FaceGuard FG-DF
P1	Monitoring + alerting on attack signals, friction and score drift; review queue with outcome labels	all	FaceGuard metrics / dashboards
P2, month 2–3	NFC chip portrait (eMRTD) as the 1:1 reference; morph and document forensics; 1:N dedupe	AV-15, AV-16	Vendor/product
P2	Native-only capture with attestation for high-risk actions (large payouts, credential resets)	AV-08, AV-09	Product
P2	Independent injection testing (CEN/TS 18099-aligned) plus a recurring red-team with current tooling	all	External lab

---

6. What the harness validated

The following were reproduced in software and measured in Deliverable 2: AV-01, AV-02 (simulated print/replay), AV-04 (naive + stealth virtual camera), AV-05 (real-time swap with pipeline latency), AV-06, AV-07 (GAN-style synthetic), AV-08 (emulator / hooked device), AV-09 (challenge-aware white-box, both a re-lit replay and a re-lit real-time face swap), and the logic attacks AV-10, AV-11, AV-12, AV-13 and AV-16 (automated API tests). AV-03 (physical masks), AV-15 (document attacks) and AV-19 (coercion) need physical or process testing and are listed as such.

---

7. Engagement checklist (applied to a client's real flow)

- **Code review:** where the verdict is read; which response fields are checked; token issuance and consumption; retry handling; fallback branches; what is logged.
- **Config:** FaceTec SDK/server versions; device-key handling; minimum SDK version; web vs native traffic split.
- **Logs, last 90 days:** failure-reason distribution, attempts per identity/device, identities per device, camera labels and UA strings on web, geo/ASN clusters.
- **Live tests (with written authorisation):** OBS virtual camera with a consented test subject; rooted Android with a camera hook; replay of a captured submission; client-claim tampering; rapid retries.
- **Outputs:** a confirmed version of the tables above, each finding with evidence and a CVSS-like severity, and the remediation plan.

References

- **[R1]** FaceTec, *ENISA RFI: Remote ID attacks & FaceTec countermeasures* (interview minutes, 3 June 2021): https://facetec.com/ENISA_RFI_Remote_ID_Attacks_FaceTec_Countermeasures.pdf
- **[R2]** iProov, *CEN/TS 18099 standard proves injection attack resilience* (Feb 2026): https://www.iproov.com/blog/cen-ts-18099-standard-proves-injection-attack-resilience
- **[R3]** CEN/TS 18099:2024, *Biometric data injection attack detection*: https://www.evs.ee/en/cen-ts-18099-2024
- **[R4]** ISO/IEC 30107-3, *Biometric presentation attack detection — Part 3: Testing and reporting*.
- **[R5]** FaceTec, *Liveness Security Report*: https://www.facetec.com/FaceTec_Liveness_Security_Report.pdf

Deliverable 2 — Adversarial Test Report

Deepfake, face-swap, injection and presentation attacks: pass/fail rates

Systems under test	B0 vendor liveness only (stand-in) · B1 + device/telemetry rules · B2 + capture-integrity model · **B3 FaceGuard (all layers)**
Campaign	1083 bona fide sessions (361 subjects) + 3610 attack sessions, 10 attack species × 361
Subjects	Held-out test split. No test face was used for training or threshold calibration
Metrics	ISO/IEC 30107-3 vocabulary: APCER per species, worst-species APCER, BPCER; DET, EER, BPCER@APCER; IAPMR for face swaps; 95 % Wilson intervals
Policy	`cal-20261014-val1083` , calibrated on the validation split's bona fide sessions only
Generated	2026-10-14T08:23:14+00:00 from `reports/results.json (make eval report)`

Headline

	Baseline B0	**FaceGuard B3**
Mean APCER across 10 attack species	87.8%	**6.4%**
Worst species	98.9% (Challenge-aware face swap (white-box))	**32.1%** (Challenge-aware replay (white-box))
Bona fide rejected (BPCER)	1.75%	**2.40%**
Bona fide sent to step-up / manual review	0.0% / 0.0%	4.0% / 0.5%
Logic attacks blocked (API tests)	–	**11/11**

---

1. Test setup

1.1 System under test

- **Vendor stand-in (B0).** No FaceTec licence is used in this portfolio project. The vendor is modelled by a frame-level PAD model (colour-texture + spectrum, trained on print/replay), the way a presentation-attack-only liveness check behaves. Its threshold is set at 1 % BPCER on validation. **FaceTec's 3D liveness is expected to be much stronger than this stand-in on print/replay** (it is iBeta-tested). The B0 rows that matter here are the **injection** rows, where the camera is bypassed and any camera-side PAD sees a clean, genuine-looking face.
- **FaceGuard (B1–B3)** adds its layers *on top of* the vendor decision, through the same `fusion.decide()` function the API calls. The evaluated policy is the deployed policy.

1.2 Bona fide corpus

1,994 public face photos (two MIT-licensed GitHub repositories) were processed: YuNet detection → 128 × 128 context crop + 5 landmarks → near-duplicate removal (64-bit dHash, Hamming ≤ 4, 173 removed) → 1,803 faces → deterministic 60/20/20 split. Each bona fide *session* is simulated through the full capture chain: the screen's challenge light reflected by the face (more than by the background), head motion, one of two camera ISP profiles (noisy, or denoised), JPEG, natural frame jitter with 4 % dropped

frames, and realistic telemetry. 3 % of honest web users run benign virtual cameras (NVIDIA Broadcast, mmhmm), 1 % have webcams that ignore the requested resolution, and 3 % of mobile devices have no attestation token. These are *not* idealised users.

1.3 Attack catalogue (PAI species / injection instruments)

Species	Family	Instrument (PAI / IAI)	Delivery (method)	Seen in training	Attack potential
Print photo	presentation	Printed photo of victim	Held in front of a real camera	yes	low
Screen replay	presentation	Photo/video of victim on a phone or monitor	Held in front of a real camera	yes	low
Virtual cam · pre-recorded	injection	Pre-recorded genuine video of victim	Virtual camera (OBS/ManyCam), driver name visible	yes	low
Virtual cam · stealth	injection	Pre-recorded genuine video of victim	Virtual camera renamed to a real webcam; added sensor-like noise and timing jitter	**no** (generalisation)	moderate
Virtual cam · offline face swap	injection	Offline deepfake (victim face swapped onto attacker video)	Virtual camera	yes	moderate
Virtual cam · real-time face swap	injection	Real-time face swap (DeepFaceLive-style) of attacker's live camera	Virtual camera; label spoofed in 50% of attempts	**no** (generalisation)	high
Virtual cam · synthetic (GAN)	injection	Fully synthetic face (GAN-style upsampling artefacts)	Virtual camera; label spoofed in 50% of attempts	**no** (generalisation)	moderate
Emulator / hooked device	injection	Pre-recorded genuine video of victim	Android emulator / rooted device with camera hook (VCAM-style module)	yes	moderate
Challenge-aware replay (white-box)	adaptive	Pre-recorded genuine video of victim, re-lit in real time	Stealth virtual camera + screen capture of the challenge, 2-5 frame pipeline lag	**no** (generalisation)	high
Challenge-aware face swap (white-box)	adaptive	Real-time face swap of a live attacker, re-lit in real time	Stealth virtual camera + screen capture of the challenge, 2-5 frame pipeline lag	**no** (generalisation)	high

Physical 3D masks (AV-03), document attacks (AV-15) and coerced users (AV-19) cannot be meaningfully simulated in software and are excluded. They need iBeta-style lab testing and process controls.

1.4 Protocol

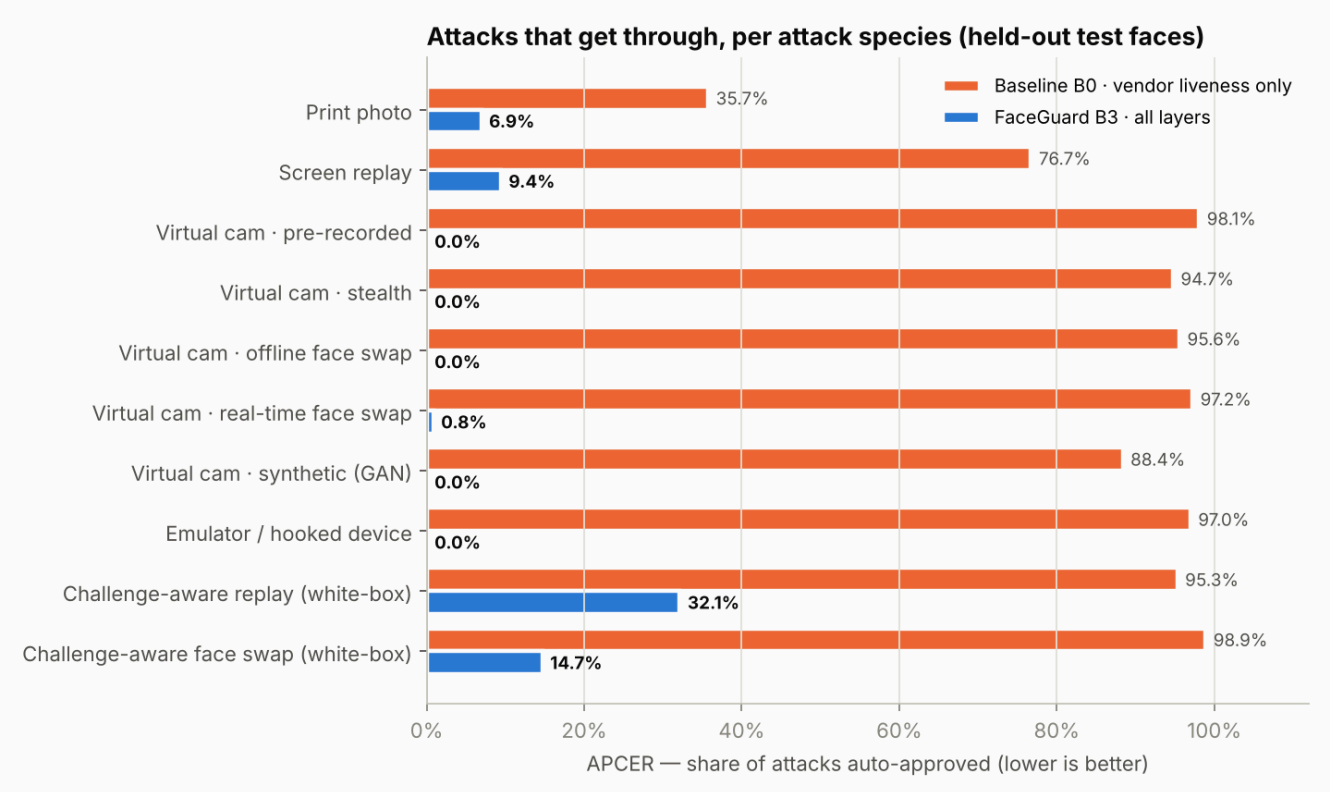
1. `src.evaluation.harness` generates seeded sessions per split: 30 frames at 30 fps, a fresh 10-slot colour challenge per session, telemetry and timestamps.
2. The models are trained on the *train* split (deepfake models: bona fide + SBI only).
3. `src.evaluation.run_eval` scores *val* and *test*, calibrates every threshold on val bona fide quantiles, then evaluates on test.
4. An attack **passes** if the decision is `APPROVE`. `STEP_UP`, `MANUAL_REVIEW` and `REJECT` all stop it.

---

2. Results — attack pass rates (APCER)

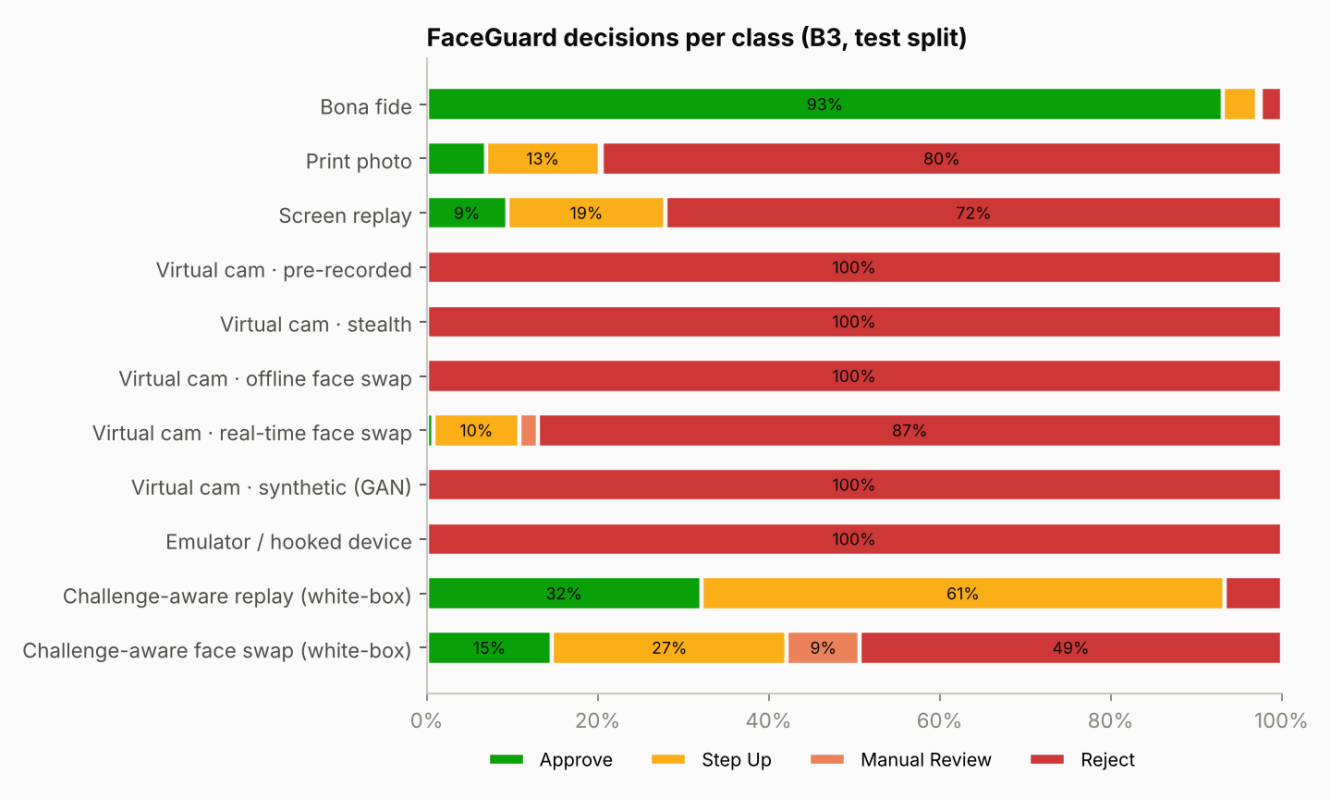
Attack species	B0 vendor only	B1 + device rules	B2 + capture integrity	B3 FaceGuard	95% CI (B3)	Verdict (B3)
Print photo	35.7%	34.3%	6.9%	**6.9%**	4.7%–10.0%	⚠️ PARTIAL
Screen replay	76.7%	72.9%	9.4%	**9.4%**	6.8%–12.9%	⚠️ PARTIAL
Virtual cam · pre-recorded	98.1%	0.0%	0.0%	**0.0%**	0.0%–1.1%	✅ PASS
Virtual cam · stealth †	94.7%	92.8%	0.0%	**0.0%**	0.0%–1.1%	✅ PASS
Virtual cam · offline face swap	95.6%	0.0%	0.0%	**0.0%**	0.0%–1.1%	✅ PASS
Virtual cam · real-time face swap †	97.2%	45.7%	1.9%	**0.8%**	0.3%–2.4%	✅ PASS
Virtual cam · synthetic (GAN) †	88.4%	42.4%	0.0%	**0.0%**	0.0%–1.1%	✅ PASS
Emulator / hooked device	97.0%	5.5%	0.0%	**0.0%**	0.0%–1.1%	✅ PASS
Challenge-aware replay (white-box) †	95.3%	93.9%	32.1%	**32.1%**	27.5%–37.1%	❌ FAIL
Challenge-aware face swap (white-box) †	98.9%	97.8%	31.6%	**14.7%**	11.4%–18.7%	❌ FAIL
Mean / worst	87.8% / 98.9%	48.5% / 97.8%	8.2% / 32.1%	**6.4% / 32.1%**		

† not seen in training by any model. Verdict on B3: ✅ PASS ≤ 2 % · ⚠️ PARTIAL ≤ 10 % · ❌ FAIL > 10 %. 95 % Wilson intervals: with 361 attempts per species, "0 %" means "below ~1 %", not "never".



2.1 Decision distribution under FaceGuard (B3)

Attack species	Approve (APCER)	Step-up	Manual review	Reject
Print photo	6.9%	13.3%	0.3%	79.5%
Screen replay	9.4%	18.6%	0.0%	72.0%
Virtual cam · pre-recorded	0.0%	0.0%	0.0%	100.0%
Virtual cam · stealth	0.0%	0.0%	0.0%	100.0%
Virtual cam · offline face swap	0.0%	0.0%	0.0%	100.0%
Virtual cam · real-time face swap	0.8%	10.0%	2.2%	87.0%
Virtual cam · synthetic (GAN)	0.0%	0.0%	0.0%	100.0%
Emulator / hooked device	0.0%	0.0%	0.0%	100.0%
Challenge-aware replay (white-box)	32.1%	61.2%	0.0%	6.6%
Challenge-aware face swap (white-box)	14.7%	27.4%	8.6%	49.3%



2.2 Red-team iteration: a finding and its fix

The first run exposed a gap. The capture-integrity model had only been trained on species *without* a delayed reflection, so real-time swaps and the challenge-aware attacker (whose reflection arrives 3–10 frames late) often scored below its review threshold, and the explicit lag check existed only as an explanation string. The fix promotes that physical check to a decision rule: `CHALLENGE_REFLECTION_DELAYED` (soft → step-up) fires when the reflection peaks after the camera window and correlates ≥ 0.4, more than 0.15 above the in-window peak. Both constants were sized on the **validation** split (0 % of bona fide sessions trigger it). Both policies on the final campaign:

B3 FaceGuard, same test campaign	Run 1 policy (capture-integrity model only)	Run 2 policy (+ delayed-reflection rule)
APCER · Challenge-aware replay (white-box)	90.9%	**32.1%**
APCER · Challenge-aware face swap (white-box)	39.9%	**14.7%**
APCER · Virtual cam · real-time face swap	8.9%	**0.8%**
Mean APCER (10 species)	15.6%	**6.4%**
Worst-species APCER	90.9%	**32.1%**
Bona fide rejected / friction	2.40% / 6.8%	2.40% / 6.8%

What remains is the challenge-aware attacker whose pipeline lag falls *inside* the honest camera window (2 frames ≈ 67 ms). In this simulation that attacker is physically indistinguishable from a live user. It is the documented residual risk, closed by native capture with attestation (Deliverable 4). For the **face-swap** variant of that attacker, the deepfake layer is the last barrier: it cuts the pass rate from 31.6% (B2) to 14.7% (B3). This is the clearest measured case for keeping a manipulation detector *behind* the challenge. *Disclosure: the gap was found on an earlier test run. The rule's constants never saw test data, and the final campaign was regenerated afterwards, with a new white-box species added (challenge-aware face swap).*

---

3. Bona fide impact (BPCER and friction)

Variant	Approved	Step-up	Manual review	Rejected (BPCER)	Friction (not auto-approved)
B0 — Vendor liveness only (PAD stand-in)	98.2%	0.0%	0.0%	1.8% [1.1%–2.7%]	1.8%
B1 — + device / telemetry integrity rules	95.6%	2.7%	0.0%	1.8% [1.1%–2.7%]	4.4%
B2 — + capture-integrity model (nonce challenge, timing, sensor noise)	94.0%	4.0%	0.0%	2.0% [1.3%–3.1%]	6.0%
B3 — + deepfake detector = FaceGuard full	93.2%	4.0%	0.5%	2.4% [1.6%–3.5%]	6.8%

Friction is the price of defence. Most of it comes from *soft* findings on honest users (benign virtual cameras, missing attestation, resolution mismatches) and from review bands. Those users are **stepped up** (retry in the native app, NFC document read), not rejected.

---

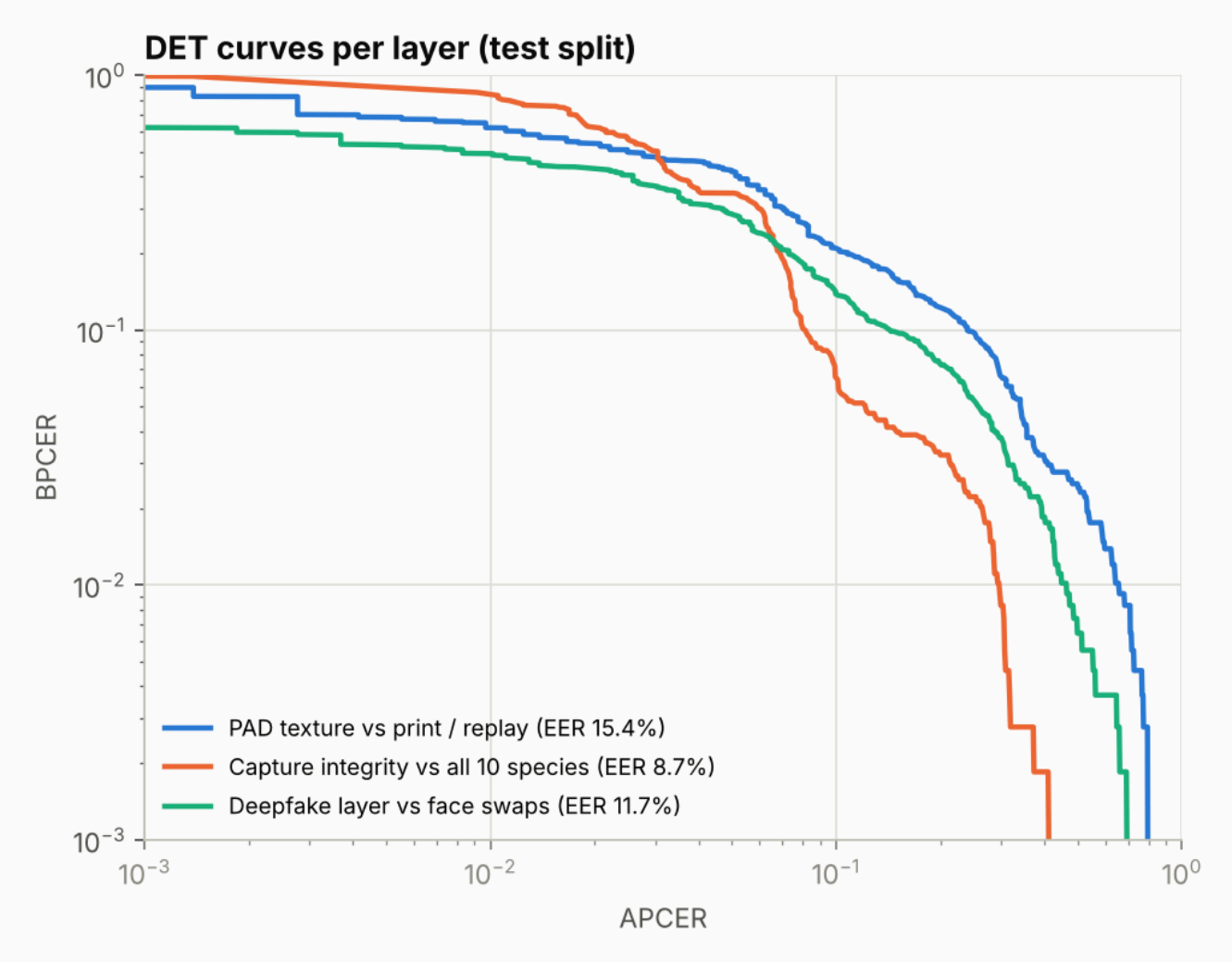
4. Per-layer analysis

Layer	Evaluated against	AUC	EER	BPCER @ APCER 5%	BPCER @ APCER 1%
PAD texture (frame)	Print photo, Screen replay	0.922	15.4%	41.6%	61.5%
Capture integrity (session)	all 10 species	0.955	8.7%	34.5%	85.6%

Layer	Evaluated against	AUC	EER	BPCER @ APCER 5%	BPCER @ APCER 1%
Deepfake layer (mean of the two below)	Virtual cam · offline face swap, Virtual cam · real-time face swap, Challenge-aware face swap (white-box)	0.949	11.7%	28.3%	48.5%
↳ CNN (PyTorch → ONNX)	Virtual cam · offline face swap, Virtual cam · real-time face swap, Challenge-aware face swap (white-box)	0.955	11.2%	27.1%	51.1%
↳ Forensics GBM	Virtual cam · offline face swap, Virtual cam · real-time face swap, Challenge-aware face swap (white-box)	0.901	19.4%	42.0%	55.9%

Per-species separability (AUC, bona fide vs. that species):

Attack species	PAD AUC	Capture-integrity AUC	Deepfake AUC (CNN / forensics)
Print photo	0.965	0.988	0.554 (0.577 / 0.525)
Screen replay	0.879	0.994	0.485 (0.467 / 0.489)
Virtual cam · pre-recorded	0.590	1.000	0.781 (0.722 / 0.793)
Virtual cam · stealth †	0.686	1.000	0.338 (0.324 / 0.360)
Virtual cam · offline face swap	0.758	1.000	0.927 (0.923 / 0.887)
Virtual cam · real-time face swap †	0.639	0.979	0.961 (0.970 / 0.909)
Virtual cam · synthetic (GAN) †	0.777	1.000	0.570 (0.543 / 0.558)
Emulator / hooked device	0.580	1.000	0.786 (0.730 / 0.792)
Challenge-aware replay (white-box) †	0.635	0.790	0.325 (0.309 / 0.353)
Challenge-aware face swap (white-box) †	0.627	0.795	0.961 (0.971 / 0.906)



Reading the layers together:

- **PAD texture** separates print better than screen replay. Single-frame texture is the weakest layer and is not designed to see injections.
- **Capture integrity** separates almost every species, because the nonce challenge cannot be pre-recorded. Its weakest species is the white-box *challenge-aware* attacker, by design.
- **Deepfake (CNN + forensics)** is trained without any deepfake. It separates face-swap species it never saw, but not fully synthetic GAN-style faces: whole-frame synthesis has no blending seam and no camera/generated noise mismatch. The capture-integrity layer covers that species instead. This is why the layers are fused.

4.1 Face matching does not stop face swaps (IAPMR)

SFace 1:1 matching (OpenCV Zoo, cosine threshold 0.363) was run between a frame of each face-swap attack and the **victim's** reference photo:

Species	n	IAPMR (matcher accepts the attack as the victim)	Matches the attacker instead
Virtual cam · offline face swap	359	89.7% [86.1%–92.4%]	6.1%
Virtual cam · real-time face swap	360	91.1% [87.7%–93.6%]	4.4%

Zero-effort impostor acceptance for context: 2.0% [1.0%–4.0%] (n = 357). The reference here is the victim's source photo, so this is an **upper bound**: with an independent ID-document portrait, IAPMR is lower. The conclusion still stands: **once a swap reaches the matcher, identity checks help the attacker**, so liveness and deepfake detection must sit in front of matching.

---

5. Logic and integration attacks (automated, `pytest`)

Logic attack (vulnerability)	Automated test	Result
AV-12 API replay of a captured submission	`test_session_replay_is_refused`	✅ blocked
Forged / guessed session token	`test_forged_session_token_is_refused`	✅ blocked
AV-10 Client-side verdict tampering (`livenessPassed: true`) + virtual camera	`test_virtual_camera_injection_rejected_even_with_client_claims`	✅ blocked
AV-11 Only `success` checked — replay check failing	`test_facetec_success_flag_alone_is_not_enough`	✅ blocked
AV-11 Vendor response missing security checks	`test_facetec_missing_checks_fail_closed`	✅ blocked
Vendor outage must not approve (fail closed)	`test_facetec_outage_fails_closed`	✅ blocked
Vendor session token reuse / payload replay	`test_simulated_vendor_enforces_one_time_token_and_replay`	✅ blocked
AV-13 Probing: >3 attempts per identity / 24 h	`test_identity_velocity_limit`	✅ blocked
AV-16 Account farming: one device, many identities	`test_device_used_by_many_identities_is_blocked`	✅ blocked
AV-04 Stealth injection (renamed driver) caught by the nonce challenge	`test_stealth_injection_caught_by_challenge`	✅ blocked
Malformed capture (frame count ≠ challenge)	`test_wrong_frame_count_is_422`	✅ blocked

These are the integration flaws from the vulnerability assessment (AV-10…AV-16). They need no ML, and they run in CI on every commit.

---

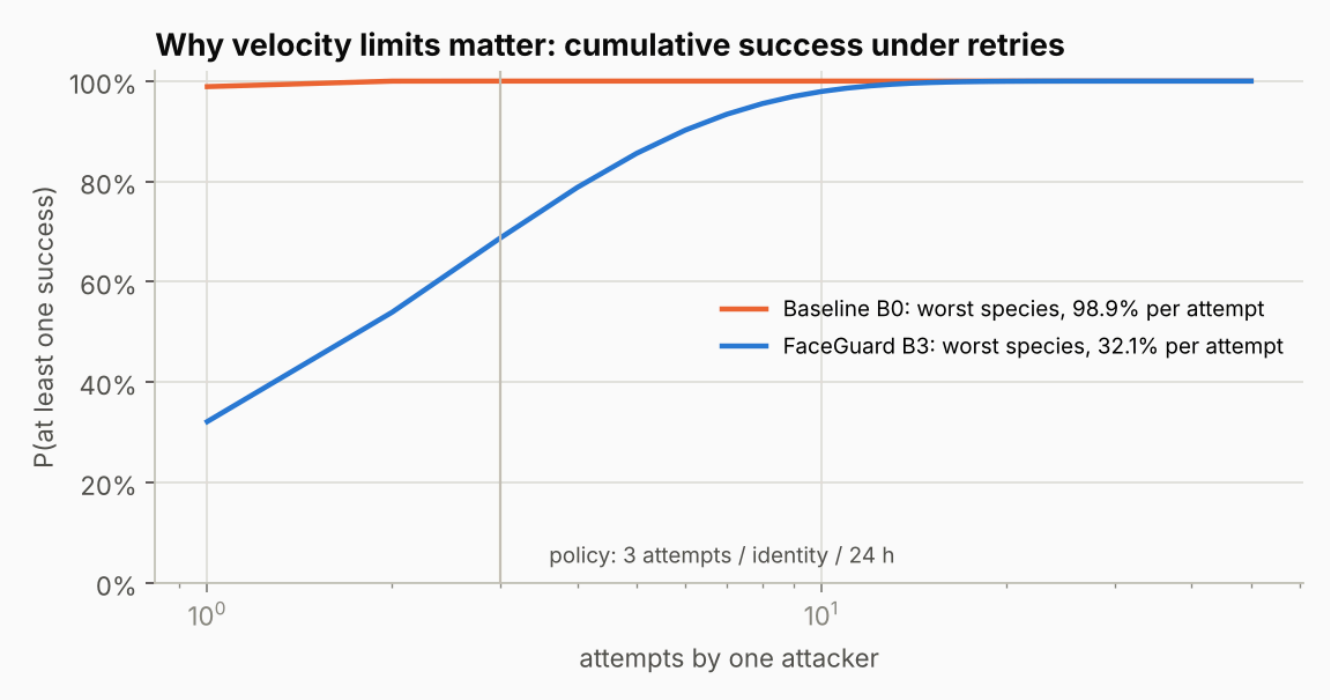
6. Retry / probing risk

With per-attempt success *p*, the chance an attacker succeeds at least once in *n* tries is $1 - (1 - p)^n$. Using the worst species' per-attempt APCER:

Attempts	B0 (p = 98.9%)	FaceGuard B3 (p = 32.1%, Challenge-aware replay (white-box))
3 (policy limit / identity / 24 h)	100.0%	68.7%
10	100.0%	97.9%

Attempts	B0 (p = 98.9%)	FaceGuard B3 (p = 32.1%, Challenge-aware replay (white-box))
50 (no velocity limit)	100.0%	100.0%

Velocity limits cap the damage of moderate species (a 0.6 % per-attempt real-time swap stays at ~2 % after 3 tries), but they cannot fix a species with a high per-attempt rate. The white-box residual is closed by channel controls (native capture + attestation), not by counting attempts.



---

7. Limitations and threats to validity

- **Simulated instruments.** Print, replay, codecs, virtual-camera timing, emulator telemetry, face swaps and GAN artefacts are software models of real tooling. They exercise every code path and give reproducible, comparable numbers, but **absolute rates will differ on real attacks**. The same harness accepts real captured media. For a client this campaign would be re-run with OBS / DeepFaceLive / rooted-device captures recorded under written authorisation.
- **Vendor stand-in.** B0 is not FaceTec. Read improvements on injection rows as "beyond what any camera-side PAD can see", not as "better than FaceTec".
- **Same simulator for training and test** on seen species. Unseen species (†), held-out subjects and calibration without attack data are the safeguards. Seen-species results are optimistic.
- **Population.** Bona fide faces are internet photos (varied ethnicity, age and quality), not a controlled demographic sample. Per-demographic BPCER should be measured on client traffic.
- **Sample size.** 361 attacks per species gives ±~1–5 pp intervals. Rare-event claims (< 1 %) need larger campaigns.

---

8. Reproduce

```
make assets data sessions train eval report   # ~1.5 h on 2 CPU cores, no GPU
pytest -q --junitxml reports/junit.xml        # logic-attack table
```

Deliverable 3 — Mitigation Layer: design, implementation and measured improvement

FaceGuard is an **additional defence layer around the FaceTec flow**. It does not replace it. It runs on the ML Model Serving Platform: every detector is a versioned, registry-managed model, so it can be promoted, rolled back and monitored like any other production model.

> All numbers below are rendered from `reports/results.json` (policy `cal-20261014-val1083`, generated 2026-10-14T08:23:14+00:00). Test campaign: 1083 bona fide sessions from 361 held-out subjects, plus 3610 attack sessions (361 per species). How the campaign was built and its limitations: Deliverable 2.

---

1. Architecture



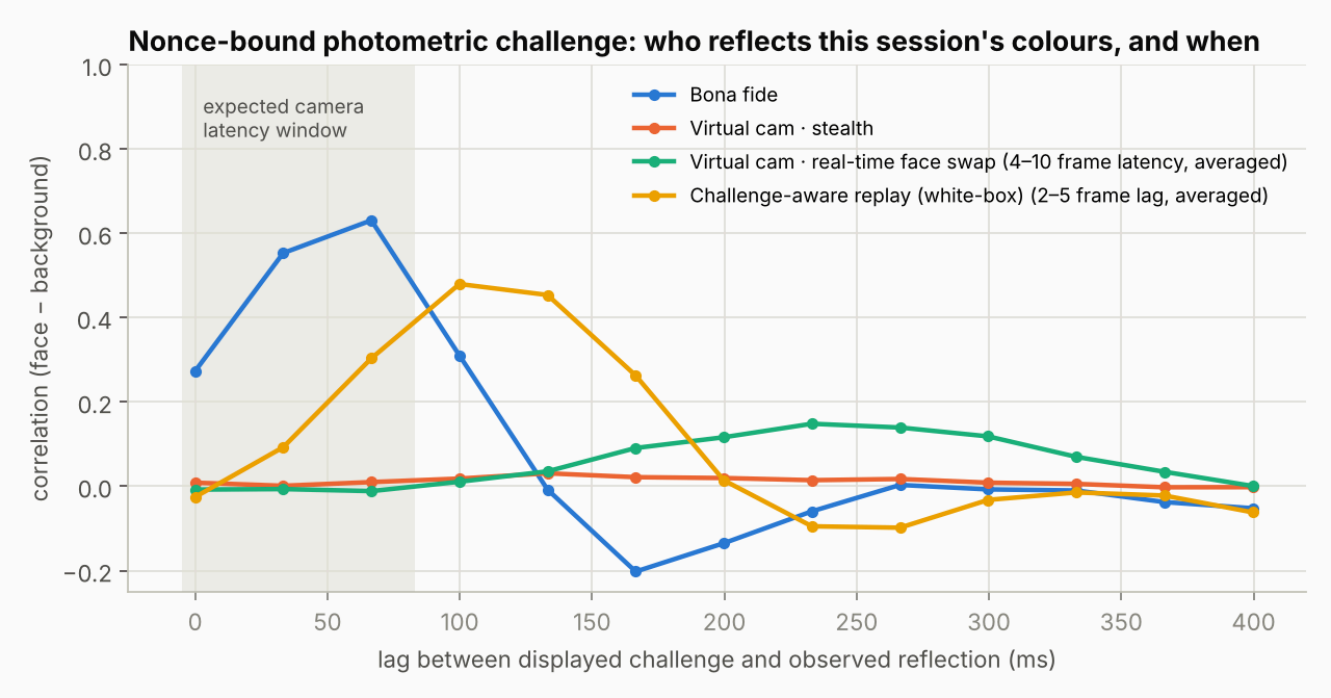
Component	Code	What it does
Session service	`src/liveness/sessions.py`	Issues a **nonce-bound colour challenge** and a one-time HMAC token (TTL 120 s). Consumption is atomic (`UPDATE … WHERE status='issued'`). Velocity limits per identity, device and identities-per-device
Vendor adapter	`src/liveness/providers/facetec.py`	Server-to-server FaceTec call. Requires **every** `faceScanSecurityChecks` flag, ignores client claims, fails closed, never logs the result blob

Component	Code	What it does
Integrity rules	`src/liveness/signals/capture_integrity.py`	Deterministic reason codes: `VIRTUAL_CAMERA`, `BROWSER_AUTOMATION`, `EMULATOR`, `HOOKING_FRAMEWORK`, `ATTESTATION_FAILED` (hard); `VIRTUAL_CAMERA_BENIGN`, `ATTESTATION_MISSING`, `SENSORS_MISSING`, `RESOLUTION_MISMATCH`, `CHALLENGE_REFLECTION_DELAYED` (soft)
Capture-integrity model	same file + `train_tabular.py`	12 stream features → HistGradientBoosting. Features: challenge correlation at camera-latency lags vs. later lags, face/background reflection ratio, frame-interval jitter, duplicated frames, sensor-noise floor, effective resolution
PAD texture model	`src/liveness/signals/texture.py`	384 features: uniform LBP on Y/Cr/Cb/H/S/V, radial log-spectrum + moiré peak, gamut/dynamic-range/glare statistics
Deepfake layer	`src/training/deepfake_net.py`, `src/liveness/signals/artifacts.py`	**Mean of two views**: a compact CNN (PyTorch → ONNX, fixed high-pass input branch) and a forensics GBM (inside/outside-face noise, sharpness, seam gradient/colour jump, up-sampling spectral peaks)
Fusion	`src/liveness/fusion.py`	Most severe outcome wins. Each layer has a *review* and a *reject* threshold calibrated from bona fide quantiles
MSP extensions	`src/serving/loader.py`, `service.py`	Loader registry (`scikit-learn/joblib`, `onnx/onnx`, `pytorch/onnx`), `ModelRuntime.get_production_model`, SQLite support for CI

1.1 The nonce-bound photometric challenge (the core anti-injection idea)

The server derives a 10-slot colour sequence from a fresh 128-bit nonce (HMAC-SHA256 keystream, no immediate repeats). During capture the client turns the screen into a light source, showing each colour for 100 ms while it records frames. FaceGuard then computes, for lags of 0–12 frames, the correlation between the challenge and the **face-minus-background** colour response:

- A **live face** is closer to the screen than the background, so it reflects more, at camera latency (≤ 2 frames).
- A **printed photo or screen** reflects uniformly or not at all (flat artefact). The face/background differential stays near zero, even though the raw face colour does follow the challenge.
- A **pre-recorded or offline-generated stream** cannot contain a sequence that did not exist when it was made.
- A **real-time face swap** does carry the reflection (the attacker sits in front of a real screen), but **late**: the swap pipeline adds 4–10 frames of latency.



Security comes from the **unpredictability of the nonce**, not from a learned model, so this layer generalises to injection tools it has never seen. Its known limit is the *challenge-aware* white-box attacker (AV-09), measured in Deliverable 2.

1.2 Deepfake layer: trained without a single deepfake

Both deepfake models are trained only on bona fide faces and **Self-Blended Images** (SBI; Shiohara & Yamasaki, CVPR 2022), made in an *injection-faithful* way: a real camera frame (sensor noise, ISP, challenge tint) has a generated, re-coloured, resampled copy of the *same* face blended in, and is delivered without a second camera pass. No face swap, GAN face or real-time swap is ever shown during training. Those species are pure generalisation tests.

An earlier iteration applied the camera model *after* blending (as if the fake were re-captured). Val AUC on SBI stalled at about 0.6 and the network memorised the ~1k real training faces. Modelling the actual delivery path, plus geometric/colour jitter of the base image, fixed it. This design decision is recorded here because it is the difference between a detector that works and one that does not.

1.3 Red-team iteration (finding → fix → re-test)

Red-team run 1 showed that real-time swaps and challenge-aware attackers got through more often than expected. The capture-integrity *model* had never seen a delayed reflection, and the lag check was only an explanation. Promoting the physical check to a deterministic rule (`CHALLENGE_REFLECTION_DELAYED` → step-up, sized on validation with 0 % bona fide hits) gave, on the same final test campaign:

B3 FaceGuard, same test campaign	Run 1 policy (capture-integrity model only)	Run 2 policy (+ delayed-reflection rule)
APCER · Challenge-aware replay (white-box)	90.9%	**32.1%**
APCER · Challenge-aware face swap (white-box)	39.9%	**14.7%**
APCER · Virtual cam · real-time face swap	8.9%	**0.8%**
Mean APCER (10 species)	15.6%	**6.4%**

B3 FaceGuard, same test campaign	Run 1 policy (capture-integrity model only)	Run 2 policy (+ delayed-reflection rule)
Worst-species APCER	90.9%	**32.1%**
Bona fide rejected / friction	2.40% / 6.8%	2.40% / 6.8%

---

2. Calibrated thresholds (policy `cal-20261014-val1083`)

Thresholds are set on the **validation split's bona fide sessions only** (attack data is never used to pick them). Each layer gets a BPCER budget, which bounds honest-user friction by construction:

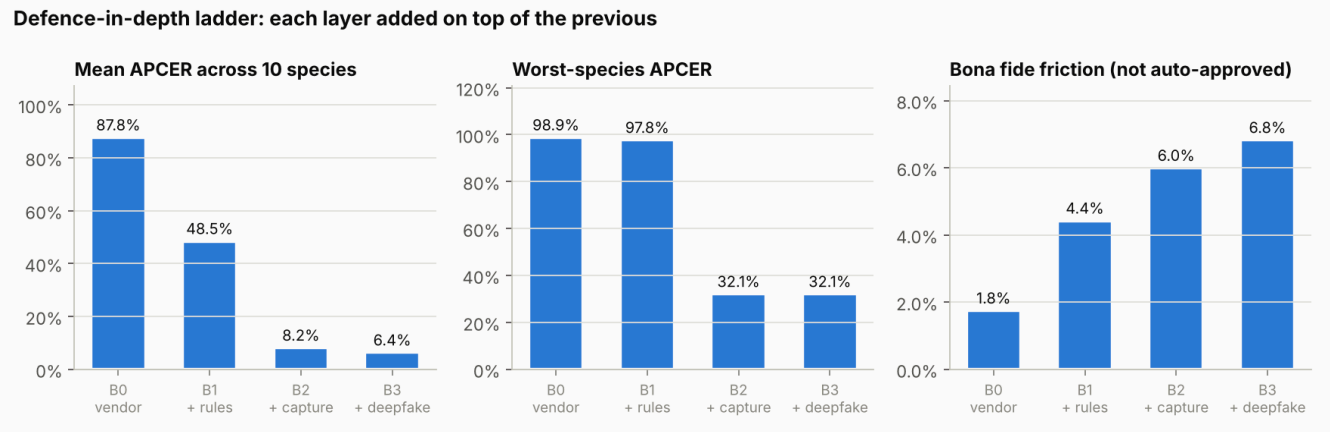
Layer	Review threshold (→ action)	Reject threshold	Bona fide budget (val)
Vendor stand-in (PAD)	–	0.9477	1.0 %
PAD texture (second opinion)	0.9477 (→ manual review)	0.9704	0.5 % reject
Capture integrity	0.9119 (→ step-up)	0.9874	2.0 % review · 0.5 % reject
Deepfake (CNN + forensics)	0.8427 (→ manual review)	0.9090	1.5 % review · 0.5 % reject

Full policy: `configs/policy.yaml`, served at `GET /v1/liveness/policy`.

---

3. Measured improvement (held-out test split)

3.1 Defence-in-depth ladder

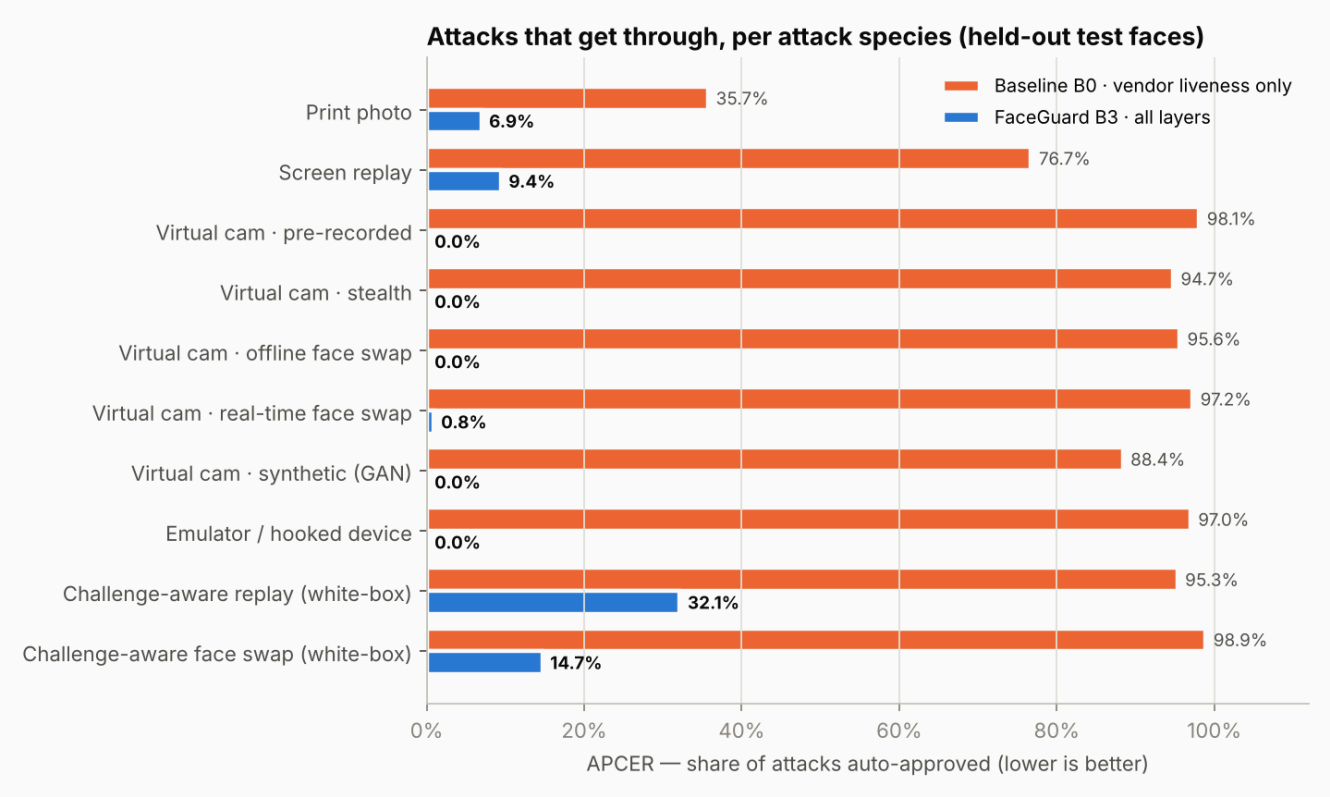


	B0 vendor only	B1 + device rules	B2 + capture integrity	B3 FaceGuard
Mean APCER (10 species)	87.8%	48.5%	8.2%	**6.4%**
Worst-species APCER	98.9%	97.8%	32.1%	**32.1%**
Bona fide rejected (BPCER)	1.75%	1.75%	2.03%	**2.40%**
Bona fide friction (step-up + review + reject)	1.8%	4.4%	6.0%	**6.8%**

3.2 Per attack species (APCER = share auto-approved)

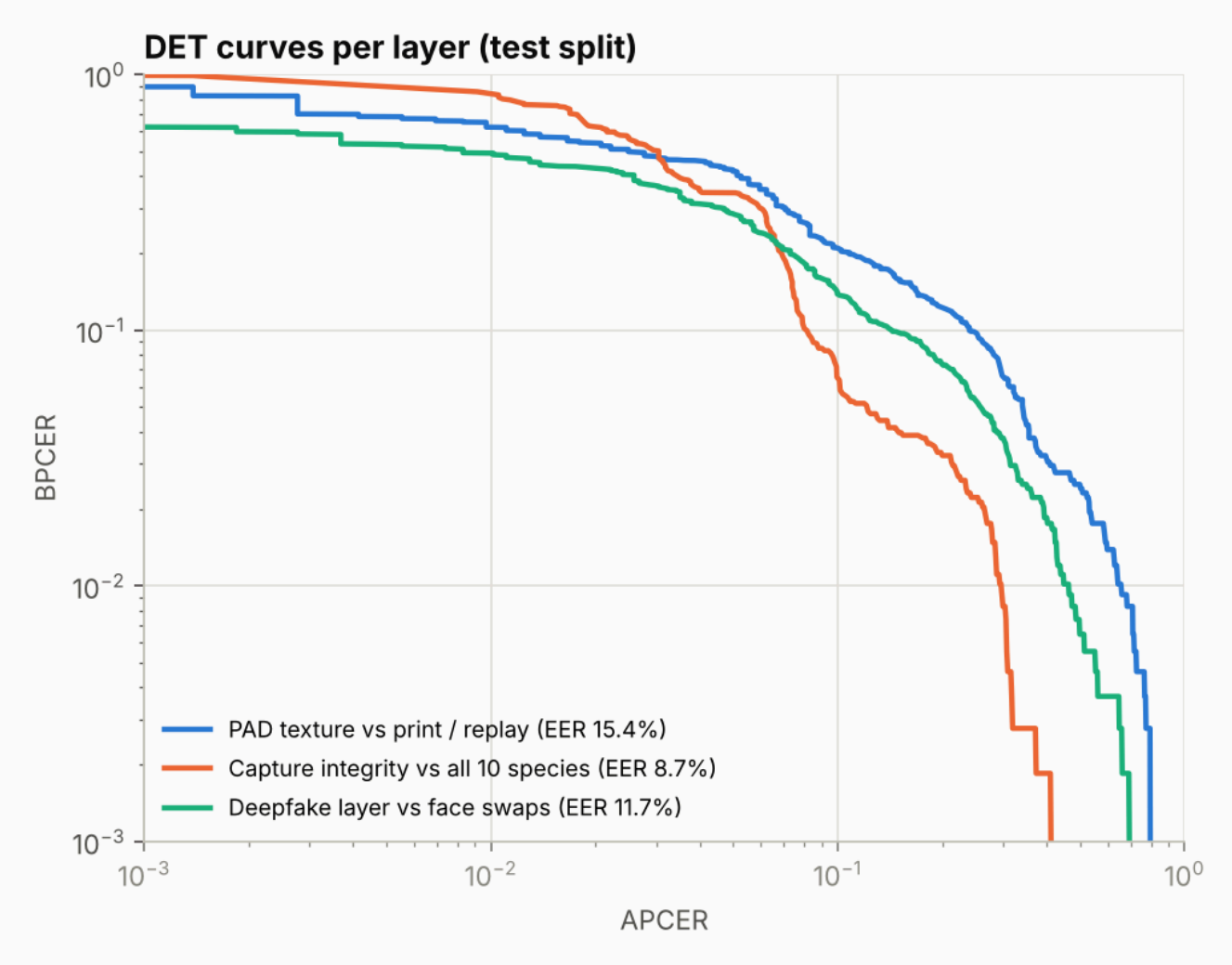
Attack species	B0 vendor only	B1 + device rules	B2 + capture integrity	B3 FaceGuard	95% CI (B3)	Verdict (B3)
Print photo	35.7%	34.3%	6.9%	**6.9%**	4.7%–10.0%	⚠️ PARTIAL
Screen replay	76.7%	72.9%	9.4%	**9.4%**	6.8%–12.9%	⚠️ PARTIAL
Virtual cam · pre-recorded	98.1%	0.0%	0.0%	**0.0%**	0.0%–1.1%	✅ PASS
Virtual cam · stealth †	94.7%	92.8%	0.0%	**0.0%**	0.0%–1.1%	✅ PASS
Virtual cam · offline face swap	95.6%	0.0%	0.0%	**0.0%**	0.0%–1.1%	✅ PASS
Virtual cam · real-time face swap †	97.2%	45.7%	1.9%	**0.8%**	0.3%–2.4%	✅ PASS
Virtual cam · synthetic (GAN) †	88.4%	42.4%	0.0%	**0.0%**	0.0%–1.1%	✅ PASS
Emulator / hooked device	97.0%	5.5%	0.0%	**0.0%**	0.0%–1.1%	✅ PASS
Challenge-aware replay (white-box) †	95.3%	93.9%	32.1%	**32.1%**	27.5%–37.1%	❌ FAIL
Challenge-aware face swap (white-box) †	98.9%	97.8%	31.6%	**14.7%**	11.4%–18.7%	❌ FAIL
Mean / worst	87.8% / 98.9%	48.5% / 97.8%	8.2% / 32.1%	**6.4% / 32.1%**		

† not seen by any model during training (generalisation test). Verdict: ✅ ≤ 2 %, ⚠️ ≤ 10 %, ❌ > 10 %.



3.3 What each layer contributes on its own

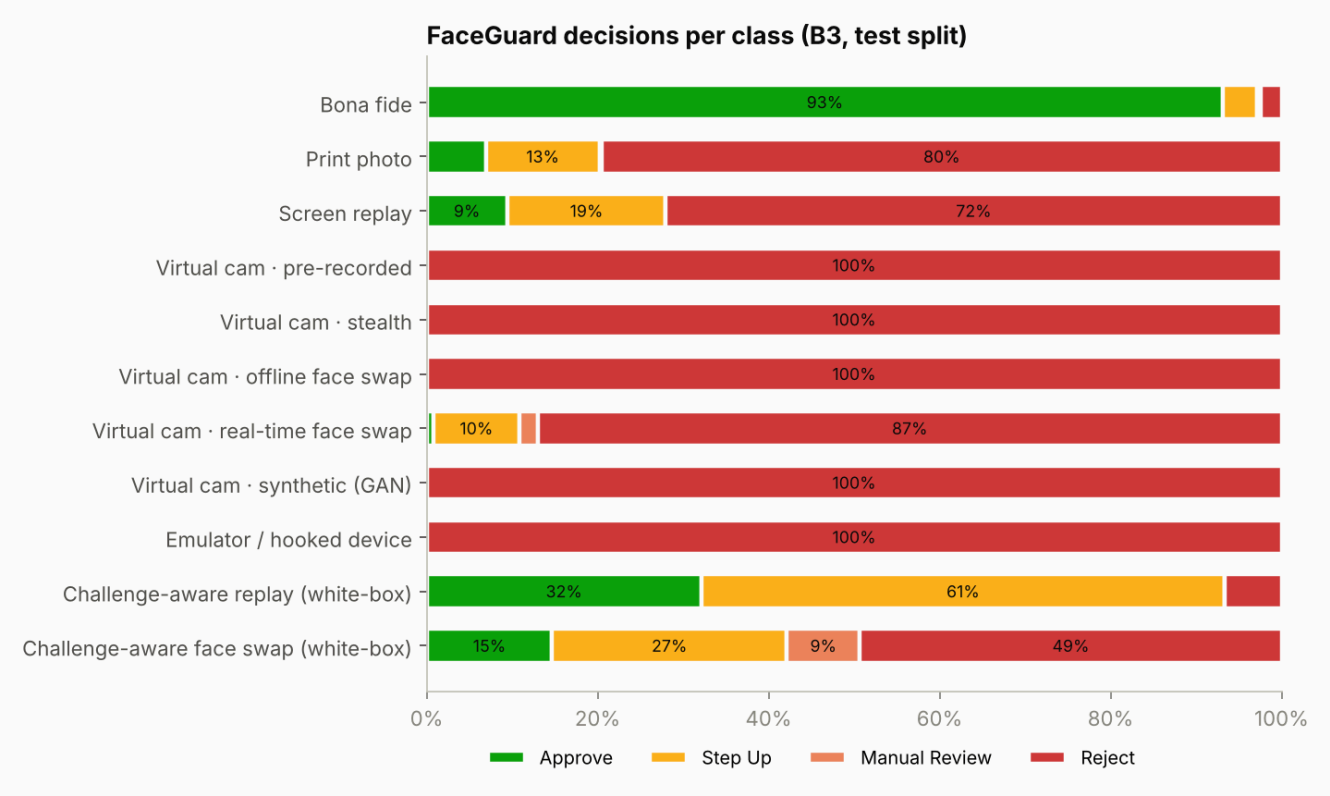
Layer	Evaluated against	AUC	EER	BPCER @ APCER 5%	BPCER @ APCER 1%
PAD texture (frame)	Print photo, Screen replay	0.922	15.4%	41.6%	61.5%
Capture integrity (session)	all 10 species	0.955	8.7%	34.5%	85.6%
Deepfake layer (mean of the two below)	Virtual cam · offline face swap, Virtual cam · real-time face swap, Challenge-aware face swap (white-box)	0.949	11.7%	28.3%	48.5%
↳ CNN (PyTorch → ONNX)	Virtual cam · offline face swap, Virtual cam · real-time face swap, Challenge-aware face swap (white-box)	0.955	11.2%	27.1%	51.1%
↳ Forensics GBM	Virtual cam · offline face swap, Virtual cam · real-time face swap, Challenge-aware face swap (white-box)	0.901	19.4%	42.0%	55.9%



3.4 Where FaceGuard sends each case

Attack species	Approve (APCER)	Step-up	Manual review	Reject
Print photo	6.9%	13.3%	0.3%	79.5%
Screen replay	9.4%	18.6%	0.0%	72.0%
Virtual cam · pre-recorded	0.0%	0.0%	0.0%	100.0%

Attack species	Approve (APCER)	Step-up	Manual review	Reject
Virtual cam · stealth	0.0%	0.0%	0.0%	100.0%
Virtual cam · offline face swap	0.0%	0.0%	0.0%	100.0%
Virtual cam · real-time face swap	0.8%	10.0%	2.2%	87.0%
Virtual cam · synthetic (GAN)	0.0%	0.0%	0.0%	100.0%
Emulator / hooked device	0.0%	0.0%	0.0%	100.0%
Challenge-aware replay (white-box)	32.1%	61.2%	0.0%	6.6%
Challenge-aware face swap (white-box)	14.7%	27.4%	8.6%	49.3%



4. Operating characteristics

- **Latency:** a 30-frame 640×480 session (face detection, 4 models, fusion) takes 162 ms p50 / 211 ms p95 of server time on a 2-vCPU container with a warm model cache (`scripts/bench_latency.py`). It is exported as `faceguard_pipeline_latency_seconds`, with an alert at p95 > 1.5 s.
- **Serving footprint:** no PyTorch in the serving image. The CNN runs on ONNX Runtime (0.67 M params).
- **Lifecycle:** each detector is an MSP model (`faceguard-pad-texture`, `faceguard-capture-integrity`, `faceguard-deepfake`, `faceguard-deepfake-forensics`). `POST /models/{name}/versions/{v}/promote` switches the version the pipeline uses on the next request (covered by an automated test). Promotion re-loads the artifact and refuses corrupt files, which is MSP behaviour.
- **Explainability:** every decision stores per-layer scores, bands, reason codes, model versions and policy version. Reviewers see `CHALLENGE_REFLECTION_DELAYED`, `VIRTUAL_CAMERA (OBS Virtual Camera)`, `cnn=0.93 forensics=0.71`, not just a number.

5. Residual risk after mitigation

- **Challenge-aware white-box injection (AV-09)** remains the hardest case. In the harness, 32.1% of re-lit replays and 14.7% of re-lit face swaps are auto-approved: those whose pipeline lag falls inside the honest camera window. The deepfake layer is what holds the face-swap variant (31.6% → 14.7%). Closing the rest needs **native capture with platform attestation** for high-risk actions, randomised challenge timing, and a physically richer challenge check (specular highlights on the eyes, geometry-consistent shading). See Deliverable 4.
- **Fully synthetic faces** are caught by the capture-integrity layer, not the deepfake layer: SBI-style training does not generalise to whole-frame synthesis (deepfake-layer AUC on that species is reported in Deliverable 2). If synthetic-identity onboarding is in scope, add a dedicated synthetic-image detector trained on current generators.
- Absolute rates come from a software-simulated campaign. Re-measuring on the client's own traffic and on real tooling is part of the engagement plan (Deliverable 4 §5).

Deliverable 4 — Recommendations: thresholds, monitoring, manual review and fallback rules

This is the operating manual for the defended flow. Each recommendation is tied to a measured result (Deliverable 2) or to a vulnerability (Deliverable 1). Machine-readable versions ship with the code: `configs/policy.yaml` (thresholds) and `monitoring/prometheus/alerts.yml` (alerts).

---

1. Thresholds

1.1 How to set them

1. **Budget BPCER per layer, not per system.** Each layer gets a share of the honest-user friction budget, and its thresholds are the matching **bona fide quantiles** on a validation set. Attack data is *not* used to set thresholds: attack mixes change weekly, honest users do not.
2. **Two thresholds per layer.** *Review* sends the case to a human or a stronger channel. *Reject* is for near-certain evidence. Only hard integrity findings (driver signatures, emulator, hooks, failed attestation) and above-reject scores auto-reject.
3. **Recalibrate on every model promotion and every capture-client release.** Thresholds are per-model-version artefacts. `src.evaluation.run_eval` regenerates them, and `policy.version` is stored with every decision.
4. **Per-platform thresholds after go-live.** Web and native have different camera pipelines (challenge latency, noise floors). Start global, split once each platform has ≥ 2,000 labelled bona fide sessions.

1.2 Recommended starting values (policy `cal-20261014-val1083`)

Layer	Review → action	Reject	BPCER budget on val bona fide (review / reject)
Vendor liveness (FaceTec)	–	vendor decision + **all** security checks	non-negotiable
PAD texture (second opinion on the audit image)	0.9477 → manual review	0.9704	1.0 % / 0.5 %
Capture integrity	0.9119 → **step-up**	0.9874	2.0 % / 0.5 %
Deepfake (CNN + forensics mean)	0.8427 → **manual review**	0.9090	1.5 % / 0.5 %
Delayed challenge reflection (rule)	→ **step-up**	–	0 % on val (peak after the camera window, corr ≥ 0.4, +0.15 over in-window)
Face match (SFace cosine)	–	< 0.363	OpenCV Zoo operating point; identity only, never liveness

Measured on the held-out test split with these values: **BPCER 2.40% · friction 6.8% · mean APCER 6.4%** (Deliverable 2).

> These numbers are calibrated on the simulated campaign. **On the client's flow, run a 2-week shadow mode first** (§5): log scores without acting on them, then set thresholds from real bona fide

> quantiles.

1.3 Integrity rules (deterministic)

Code	Severity	Action	Note
`VIRTUAL_CAMERA` (OBS, ManyCam, XSplit, e2eSoft, DroidCam, v4l2loopback…)	hard	REJECT	Keep the pattern list in config. Review new labels weekly from metrics
`VIRTUAL_CAMERA_BENIGN` (NVIDIA Broadcast, mmhmm, Camo)	soft	STEP_UP	Honest users. Ask them to switch camera rather than rejecting
`BROWSER_AUTOMATION` (`navigator.webdriver`)	hard	REJECT	
`EMULATOR`, `HOOKING_FRAMEWORK`	hard	REJECT	Mobile: verify on the **server** from the attestation payload, not client flags
`ATTESTATION_FAILED` (Play Integrity / App Attest)	hard	REJECT	Require `MEETS_DEVICE_INTEGRITY` (or stronger) for onboarding
`ATTESTATION_MISSING`, `SENSORS_MISSING`, `RESOLUTION_MISMATCH`	soft	STEP_UP	~3 % of honest mobile devices cannot attest

---

2. Manual review and step-up rules

2.1 Routing

Trigger	Route	SLA	Reviewer sees
Deepfake score in review band	**Manual review** (fraud analyst)	4 h (onboarding), 15 min (high-value login)	Audit image + 3 scored frames, CNN vs forensics sub-scores and feature values, FaceTec result
PAD score in review band	Manual review	4 h	Audit image, PAD score, FaceTec result
Capture-integrity review band, or any *soft* finding	**Step-up**: retry in native app with attestation, or NFC chip read, or scheduled video call	user-driven	Reason code shown to the analyst only
Vendor unavailable	Step-up (retry later)	–	Never approve on vendor error (fail closed)
≥ 2 step-ups for one identity in 24 h	Manual review	24 h	Full session history for the identity and device
Approved session, then a chargeback / ATO report on the account within 30 days	Retro-review + label	72 h	Feeds the training set

2.2 Reviewer decision rules (written guidance)

1. **Never approve on face similarity alone.** A face swap *is* the victim's face. In Deliverable 2, SFace accepted the victim in 91.1% [87.7%–93.6%] of real-time swaps.
2. Treat these as **confirmed attack**: a hard integrity finding on any session for the same device; `CHALLENGE_REFLECTION_DELAYED` together with deepfake score > review; identical background/lighting across different identities.

3. Signs that point to **genuine**: low deepfake score from *both* sub-models, the challenge reflected at camera latency, a native device with valid attestation.
4. Unsure → **step-up**, not reject. The cost of one extra NFC read is far below the cost of a false reject plus support contact.
5. Every review outcome (`confirmed_attack` / `confirmed_genuine`) is recorded through `POST /v1/liveness/reviews/{id}`. Those labels become the next training and calibration set.

2.3 Fallbacks: always stronger, never weaker

- ❌ "Upload a selfie instead", "skip liveness and verify by email", or "try another browser" with no extra checks. Attackers route to the weakest path (AV-14).
- ✅ Native app with attestation · NFC eMRTD chip portrait as the 1:1 reference (signed by the issuing state, which also closes AV-15) · live video call with a trained agent for the residue.

2.4 Velocity and anti-probing

Limit	Value	Why
Liveness attempts per identity	3 / 24 h	At the mean per-attempt APCER (6.4%) an attacker succeeds 18.0% of the time within 3 tries vs 96.3% within 50
Attempts per device	5 / 24 h	Blocks iterating across identities
Distinct identities per device	2 / 30 days	Account farming (AV-16)
Client-side error detail	generic ("we couldn't verify you")	Reason codes only in the back-office. Never tell the attacker which layer fired
Session TTL	120 s, one-time	Limits relaying the challenge to a remote rig (AV-09, AV-12)

---

3. Monitoring

3.1 Attack signals

Alert (Prometheus)	Condition	Owner	First response
`InjectionToolingSurge`	> 5 % of sessions with injection signatures, 10 min	Fraud	Check new camera labels / ASNs; raise to step-up for the affected cohort
`ChallengeFailureSurge`	> 8 % `CHALLENGE_NOT_REFLECTED` / `…_DELAYED`, 15 min	Fraud + Mobile	Distinguish a campaign (clustered devices/IPs) from a client regression (one app build)
`RejectRateSpike`	reject share > 2× 7-day baseline, 30 min	Fraud	Same
`SessionReplayAttempt`	any replayed / forged session	Fraud	Block device, review the account
`DeviceFarmSuspected`	> 5 multi-identity device blocks / h	Fraud	Link analysis on devices and accounts

3.2 Model and quality signals

Alert	Condition	Owner	Response
`BonaFideFrictionHigh`	step-up + review > 12 % for 1 h (≈ 1.4× the calibrated 6.8%)	ML	Check the capture-client release, camera mix, per-platform score shift. Recalibrate
`DeepfakeScoreDrift`	median deepfake score > 2× 7-day median for 2 h	ML	New camera/codec in the population, or a campaign. Check review outcomes
`ReviewersOverturnMostCases`	> 90 % of reviews `confirmed_genuine` in 24 h	ML	Review band too wide. Raise the threshold or retrain
`LivenessVendorUnavailable`	> 3 vendor errors / 5 min	Platform	Flow fails closed (step-up). Page on-call
`VendorSecurityCheckFailures`	any replay / token / audit-trail check failure	Fraud	Investigate the integration or an attack
`PipelineLatencyHigh`	p95 > 1.5 s, 10 min	Platform	Scale out, check the model cache

Dashboard: `monitoring/grafana/dashboards/faceguard.json` (decision mix, friction, findings by code, per-layer score quantiles, session rejections, vendor checks, latency, detector model versions).

3.3 Weekly review (30 min)

- Top new camera labels / user agents / ASNs among rejects.
- Reviewer agreement and turnaround. Confirmed-attack rate per route.
- Per-platform and (where lawful) per-demographic BPCER from step-up outcomes.
- Retraining queue: confirmed attacks the models scored low (false accepts found by retro-review).

---

4. Model lifecycle (on MSP)

1. Train → register as a new **staging** version (`src/training/register.py`).
2. **Shadow**: score live traffic with both versions and compare score distributions and review outcomes for ≥ 7 days.
3. Recalibrate thresholds for the new version and store them as a new `policy.version`.
4. **Promote** through the MSP API. The previous version is archived and the cache invalidated. The pipeline picks up the new version on the next request.
5. **Rollback**: re-register the previous artifact as a new version and promote it. MSP blocks promoting archived versions on purpose, keeping an auditable history.

---

5. Engagement plan (to take this from the simulated campaign to the client's production flow)

Week	Activity	Output
1	Flow, code and log review against the checklist in Deliverable 1 §7. P0 integration fixes (server-side verdict, all checks, one-time tokens, velocity, fallbacks)	Confirmed findings + P0 PRs
2	FaceGuard capture add-on in **shadow mode**: scores logged, no decisions	Real bona fide score distributions

Week	Activity	Output
3	Authorised red-team with real tooling: OBS + pre-recorded video, DeepFaceLive-class swap, rooted Android + camera hook, print/screen, replayed submissions. Same harness and metrics	Real APCER per species, before/after
4	Calibrate on real bona fide quantiles, enable review/step-up routing, alerts and dashboard, reviewer guide	Live policy v1
5–8	Weekly review loop, retraining from review labels, per-platform thresholds	Policy v2 + model v1.1
Later	Independent CEN/TS 18099-aligned injection test, iBeta for any PAD changes	External attestation